README_technical

TEP Data Uploader — Technical README

Overview

TEP Data Uploader is a client-side browser application that converts spreadsheet data (XLSX) into XML files conforming to the CDN Post-TX Exchange schema (`urn:cdn:pdx:v1`), and uploads them to an Amazon S3 bucket for ingestion by TEP (The Everyone Project).

The app also provides functionality to review error files generated by TEP when ingestion fails.

There is no server-side component. The application is built with Svelte and compiled to static files (HTML/CSS/JS) that can be served from any web host or opened directly from the filesystem.

Technology Stack

- **Svelte** — component-based UI framework; compiled to vanilla JS, no runtime overhead
- **Vite** — build tooling; provides `npm run dev` (dev server) and `npm run build` (static output)
- **ExcelJS** — client-side XLSX parsing; replaces SheetJS (`xlsx@0.18.5`), which has known prototype-pollution and ReDoS vulnerabilities (GHSA-4r6h-8v6p-xvw6, GHSA-5pgg-2g8v-p4x9). ExcelJS automatically returns JavaScript `Date` objects for date-formatted cells, equivalent to SheetJS's `{ cellDates: true }` option. All XLSX calls are isolated behind `src/lib/spreadsheet.js` — to swap the library again, only that file needs to change.
- **AWS SDK for JavaScript v3 (S3 client)** — S3 operations
- **Web Crypto API** (`crypto.subtle`) — SHA-256 hashing for deduplication; browser-native, no library required
- **Levenshtein distance library** — fuzzy column header matching (small enough to inline)

Project Structure

```
src/
├── App.svelte                  ← top-level shell; renders current section and step
├── config.js                   ← COLUMN_CONFIG, RECORD_VALIDATOR
├── stores.js                   ← shared Svelte writable stores
├── steps/
│   ├── FileSelection.svelte
│   ├── SheetSelection.svelte
│   ├── ColumnMappingReview.svelte
│   ├── ValidationReport.svelte
│   ├── UploadProgress.svelte
│   └── ResultsSummary.svelte
├── lib/
│   ├── spreadsheet.js          ← XLSX adapter (swap the library here, nowhere else)
│   ├── xml.js                  ← XML generation
│   ├── validation.js           ← field-level and record-level validation
│   ├── dedupe.js               ← SHA-256 hashing and local hash cache
│   ├── s3.js                   ← all S3 operations + filename parsing
```

```
│   ├── errorCache.js           ← localStorage cache for error review state
│   ├── errorPipeline.js        ← S3 listing, batch grouping, error detail fetching
│   ├── learnedAliases.js       ← persist user-confirmed column→field mappings
│   └── debug.js                ← debug logging utility (see Debug Mode below)
└── components/
    ├── Settings.svelte         ← settings modal
    ├── StepIndicator.svelte    ← step progress bar
    ├── ErrorReview.svelte      ← error review section root
    ├── ErrorBatchList.svelte   ← batch list view
    ├── ErrorBatchCard.svelte   ← single batch summary card
    ├── ErrorBatchDetail.svelte ← batch detail view with error files
    └── ErrorFileRow.svelte     ← expandable error file row
```

Navigation and UI

Overall structure

The app has two top-level sections; no router library is needed:

- **Upload** — the main multi-step wizard
- **Error Review** — browsing TEP error reports grouped by upload batch

A persistent top navigation bar contains the app title, the two section links, and a settings icon that opens the **Settings modal**. The Settings modal can be opened at any time without interrupting the current wizard step.

Wizard navigation

The wizard is a **paged flow** — one step occupies the full viewport at a time. A `<StepIndicator>` component below the nav bar shows progress. State is held in two stores:

- `currentSection` — `'wizard'` or `'errorReview'`
- `currentStep` — integer index into the wizard step sequence

Navigation between steps is driven by explicit “Proceed” and “Back” actions within each step component. Browser history / the back button are not supported.

Where earlier-step context is useful on a later screen (e.g. seeing which columns were mapped while reviewing validation errors), a compact collapsible summary should be embedded in the later step’s component rather than requiring the user to navigate back.

Application Flow

Step 1: Credential Check

On load, the app checks `localStorage` for AWS credentials:

- `tep_aws_access_key_id`
- `tep_aws_secret_access_key`
- `tep_aws_bucket_name`
- `tep_aws_region`

If any are missing, the user is redirected to the **Settings** page where they can enter these values (provided by TEP). Credentials are persisted in `localStorage`.

Step 2: File Selection

The user selects an XLSX file. The app parses it using ExcelJS and reads the list of

sheet names. If parsing fails, an error is displayed.

This page also presents a checkbox: **“Skip column review if all columns matched OK”**.

Step 3: Sheet Selection

If the workbook contains more than one sheet, an interstitial screen is shown listing all sheet names. The user selects which sheet to process. If the workbook contains only one sheet, this step is skipped automatically.

Step 4: Header Row Detection

The app scans the **first 100 rows** of the selected sheet to find the header row. Each row is scored by counting how many of the app’s known field aliases (across all entries in `COLUMN_CONFIG`) are matched by at least one cell in that row, using Levenshtein distance ≤ 3 as the match threshold. The highest-scoring row is treated as the header row.

If no row scores above a minimum threshold (at least one mandatory field alias matched), an error is displayed.

Step 5: Column Mapping (Automatic)

For each non-empty cell in the header row:

1. Check whether the cell value matches any **learned alias** stored in `localStorage` (key: `tep_learned_aliases`) for an unassigned field. Matching is case-insensitive. If a learned alias matches, that field is assigned with a distance of 0 and marked as assigned — no Levenshtein comparison is needed.
2. If no learned alias matched, calculate the Levenshtein distance between the cell value and every alias in every unassigned field from the column configuration.
3. Assign the cell to the field with the lowest distance. If two fields tie, the one appearing earlier in `COLUMN_CONFIG` takes priority.
4. Mark that field as assigned so it cannot be matched again.

Step 6: Auto-Skip Logic

If **all** of the following are true, the column review step (Step 7) is skipped:

- The “Skip column review” checkbox was ticked.
- Every **mandatory** field was matched with a Levenshtein distance of 0 (exact match).
- Every **optional** field that was matched also had a distance of 0.

Step 7: Column Mapping Review

A review screen is presented listing all fields:

- **Mandatory fields** listed first. Unmatched mandatory fields highlighted in red.
- **Optional fields** follow. Unmatched optional fields highlighted in amber, dropdown defaulting to “Ignore this column”.
- Each field has a dropdown populated with all header row values. Optional fields also include an “Ignore this column” option.
- **Validation rules:** no two fields may map to the same spreadsheet column; all mandatory fields must have a mapping.
- A “Proceed” button is enabled only when validation passes.

When the user clicks **Proceed**, the app saves **learned aliases**: for every mapped field where the matched column header is not already in the field’s hard-coded `aliases`

array (case-insensitive), the column header is written to `localStorage` (key: `tep_learned_aliases`) as a new alias for that field. On subsequent uploads, these learned aliases are checked first at distance 0 (see Step 5), allowing identical spreadsheet structures to auto-match without user intervention.

Step 8: Field and Record Validation

Before any XML is generated or data uploaded, all data rows are validated. This step runs a full validation pass across the entire dataset and reports any problems to the user before proceeding.

Field-level validation

Each mapped field value is checked against the `type` defined in the column configuration (see **Field Types** below). Validation is applied after right-trimming string values. Unmapped optional fields that have a non-null default are validated against the resolved default value.

Record-level validation

After field-level checks pass for a row, the `RECORD_VALIDATOR` function (see **Record Validator** below) is called with the full set of resolved field values. It returns either `true` (valid) or an error string describing the problem.

Validation report

If any rows fail validation, a **Validation Report** screen is shown:

- Each failing row is listed with its spreadsheet row number.
- All mapped field values for the row are displayed.
- Invalid fields are highlighted in red, each showing the specific error message.
- The user may either:
  - **Go back** — return to the column mapping review to correct the mapping, or fix the spreadsheet and start over.
  - **Proceed, skipping invalid rows** — continue with only the valid rows. Invalid rows are counted in the results summary.

If all rows pass validation, this step is silent and the app proceeds directly to Step 9.

Step 9: XML Generation

For each valid data row:

1. String field values are right-trimmed to remove trailing whitespace. Date fields have already been converted to ISO 8601 UTC strings during validation.
2. A DOM-based approach is used: the browser's `DOMParser` and `XMLSerializer` create a well-formed XML document.
3. The template XML structure (see **XML Output Structure** below) is instantiated.
4. Each mapped field value is inserted at the location specified by its `xpath` (XPath-like notation — see **XPath-like Notation** below) in the column configuration.
5. Default values are applied for any unmapped optional fields that define a default.
6. The completed DOM is serialised to an XML string.

XML Output Structure

Each row produces a complete XML document:

```
<?xml version="1.0" encoding="UTF-8"?>
<Document
```

```
  schemaVersion="1.0"
  xmlns="urn:cdn:pdx:v1">
  <Publications>
    <Publication
      publicationId="{value}"
      episodeId="{value}"
      availabilityMode="{value}"
      isRepeat="{value}"
      isPrimary="{value}">
      <PublicationDateTime>{value}</PublicationDateTime>
      <WindowClosureDateTime>{value}</WindowClosureDateTime>
      <ChannelPlatforms>
        <ChannelPlatform label="{value}" id="{value}">
          <SubChannel label="{value}" id="{value}" isCore="true"/>
          <!-- up to 4 SubChannel elements -->
        </ChannelPlatform>
      </ChannelPlatforms>
    </Publication>
  </Publications>
</Document>
```

Elements or attributes with no value (unmapped optional fields with no default) are omitted entirely.

> **SubChannel handling:** The v1 schema requires at least one `<SubChannel>` per `<ChannelPlatform>`. If no sub-channel data is provided in the spreadsheet, the app automatically inserts a default `<SubChannel label="Network" isCore="true"/>`. If sub-channels are provided, the first one is automatically marked with `isCore="true"` to indicate the core network feed.

Step 10: Deduplication (Local Cache)

A SHA-256 hash is computed from the serialised XML string using `crypto.subtle.digest` (async). A **rolling circular buffer** of the last 1,000 hashes is maintained in `localStorage` (key: `tep_upload_hash_cache`). This is a global cache across all uploads from this browser. If the hash is found, the row is skipped.

Step 11: Upload to S3

The upload phase runs row by row, displaying a **progress bar** to the user. The progress bar tracks rows completed (checked + uploaded or skipped) against total valid rows.

For each non-duplicate row:

1. **Generate the S3 key:** `incoming/{sha256_hash}-{batch_id}-{yyyymmdd}-{hhmmss}-{random_number}.xml`
2. **Remote deduplication check:** `ListObjectsV2` with prefix to check `incoming/{sha256_hash}-`, `complete/{sha256_hash}-`, and `errors/{sha256_hash}-`. If a match is found, the row is skipped. The progress bar advances.
3. **Upload:** `PutObject` with `Content-Type: application/xml`.
4. **On failure:** Alert the user and halt. No further rows are uploaded.
5. **On success:** Add the hash to the circular buffer. The progress bar advances.

   > **Note:** Duplicate uploads are handled gracefully by TEP, so the dedup checks are an optimisation to avoid unnecessary processing rather than a hard requirement.

Step 12: Results Summary

- **X** rows uploaded successfully.
- **Y** rows skipped (duplicates).

- **Z** rows skipped (failed validation).
- If upload failure occurred: full record data for the failed row.
- **W** rows not yet processed.
- If unprocessed rows remain, a **Retry** button returns to Step 9. Previously uploaded rows will be caught by deduplication.

Column Configuration

The column configuration is a hard-coded array defining the mapping between spreadsheet columns and XML output locations. The `xpath` values use the XPath-like notation described in **XPath-like Notation** below, relative to the `Publication` element.

```
const COLUMN_CONFIG = [
  {
    fieldName: "Publication ID",
    xpath: "@publicationId",
    type: "string",
    mandatory: true,
    default: null,
    aliases: [
      "Publication ID", "PublicationID", "Publication Id",
      "Pub ID", "PubID"
    ]
  },
  {
    fieldName: "Episode ID",
    xpath: "@episodeId",
    type: "string",
    mandatory: true,
    default: null,
    aliases: [
      "Episode ID", "EpisodeID", "Episode Id",
      "Ep ID", "EpID"
    ]
  },
  {
    fieldName: "Availability Mode",
    xpath: "@availabilityMode",
    type: "enum",
    enumValues: ["broadcast", "onDemand"],
    mandatory: true,
    default: null,
    aliases: [
      "Availability Mode", "AvailabilityMode",
      "Avail Mode", "Mode"
    ]
  },
  {
    fieldName: "Is Repeat?",
    xpath: "@isRepeat",
    type: "boolean",
    mandatory: false,
    default: "true",
    aliases: [
      "Is Repeat?", "Is Repeat", "IsRepeat",
      "Repeat", "Repeat?"
    ]
  },
  {
    fieldName: "Is Primary?",
    xpath: "@isPrimary",
    type: "boolean",
    mandatory: false,
    default: "false",
```

```
    aliases: [
      "Is Primary?", "Is Primary", "IsPrimary",
      "Primary", "Primary?"
    ]
  },
  {
    fieldName: "Publication Date and Time",
    xpath: "PublicationDateTime",
    type: "datetime",
    mandatory: true,
    default: null,
    aliases: [
      "Publication Date and Time", "Publication DateTime",
      "PublicationDateTime", "TX Date", "TX DateTime",
      "Transmission Date", "Pub Date"
    ]
  },
  {
    fieldName: "Window Closure Date and Time",
    xpath: "WindowClosureDateTime",
    type: "datetime",
    mandatory: false,
    default: null,
    aliases: [
      "Window Closure Date and Time", "Window Closure DateTime",
      "WindowClosureDateTime", "Window Close",
      "Closure Date", "End Date"
    ]
  },
  {
    fieldName: "Channel Label",
    xpath: "ChannelPlatforms/ChannelPlatform/@label",
    type: "string",
    mandatory: true,
    default: null,
    aliases: [
      "Channel Label", "Channel Name", "ChannelLabel",
      "Channel", "Platform"
    ]
  },
  {
    fieldName: "Channel ID",
    xpath: "ChannelPlatforms/ChannelPlatform/@id",
    type: "string",
    mandatory: false,
    default: (row) => row["Channel Label"],
    aliases: [
      "Channel ID", "ChannelID", "Channel Id",
      "Channel Code"
    ]
  },
  {
    fieldName: "Sub-channel 1 Label",
    xpath: "ChannelPlatforms/ChannelPlatform/SubChannel[0]/@label",
    type: "string",
    mandatory: false,
    default: null,
    aliases: [
      "Sub-channel 1 Label", "SubChannel 1 Label",
      "Sub-channel 1", "Sub Channel 1 Label", "Region 1"
    ]
  },
  {
    fieldName: "Sub-channel 1 ID",
    xpath: "ChannelPlatforms/ChannelPlatform/SubChannel[0]/@id",
    type: "string",
    mandatory: false,
    default: (row) => row["Sub-channel 1 Label"],
```

```
  aliases: [
    "Sub-channel 1 ID", "SubChannel 1 ID",
    "Sub-channel 1 Id", "Sub Channel 1 ID", "Region 1 ID"
  ]
},
{
  fieldName: "Sub-channel 2 Label",
  xpath: "ChannelPlatforms/ChannelPlatform/SubChannel[1]/@label",
  type: "string",
  mandatory: false,
  default: null,
  aliases: [
    "Sub-channel 2 Label", "SubChannel 2 Label",
    "Sub-channel 2", "Sub Channel 2 Label", "Region 2"
  ]
},
{
  fieldName: "Sub-channel 2 ID",
  xpath: "ChannelPlatforms/ChannelPlatform/SubChannel[1]/@id",
  type: "string",
  mandatory: false,
  default: (row) => row["Sub-channel 2 Label"],
  aliases: [
    "Sub-channel 2 ID", "SubChannel 2 ID",
    "Sub-channel 2 Id", "Sub Channel 2 ID", "Region 2 ID"
  ]
},
{
  fieldName: "Sub-channel 3 Label",
  xpath: "ChannelPlatforms/ChannelPlatform/SubChannel[2]/@label",
  type: "string",
  mandatory: false,
  default: null,
  aliases: [
    "Sub-channel 3 Label", "SubChannel 3 Label",
    "Sub-channel 3", "Sub Channel 3 Label", "Region 3"
  ]
},
{
  fieldName: "Sub-channel 3 ID",
  xpath: "ChannelPlatforms/ChannelPlatform/SubChannel[2]/@id",
  type: "string",
  mandatory: false,
  default: (row) => row["Sub-channel 3 Label"],
  aliases: [
    "Sub-channel 3 ID", "SubChannel 3 ID",
    "Sub-channel 3 Id", "Sub Channel 3 ID", "Region 3 ID"
  ]
},
{
  fieldName: "Sub-channel 4 Label",
  xpath: "ChannelPlatforms/ChannelPlatform/SubChannel[3]/@label",
  type: "string",
  mandatory: false,
  default: null,
  aliases: [
    "Sub-channel 4 Label", "SubChannel 4 Label",
    "Sub-channel 4", "Sub Channel 4 Label", "Region 4"
  ]
},
{
  fieldName: "Sub-channel 4 ID",
  xpath: "ChannelPlatforms/ChannelPlatform/SubChannel[3]/@id",
  type: "string",
  mandatory: false,
  default: (row) => row["Sub-channel 4 Label"],
  aliases: [
```

```
      "Sub-channel 4 ID", "SubChannel 4 ID",
      "Sub-channel 4 Id", "Sub Channel 4 ID", "Region 4 ID"
    ]
  }
];
```

Field Types

The `type` property controls how a field value is validated in Step 8 and how it is extracted from the spreadsheet. ExcelJS automatically returns JavaScript `Date` objects for date-formatted cells, so no special configuration is required.

Type	Validation rule	Value used in XML
`"string"`	Any non-empty string (mandatory) or any string (optional), after trimming.	The trimmed string value.
`"datetime"`	The cell must be either a Date object (ExcelJS returns native `Date` for date-formatted cells) or a text cell containing a strict ISO 8601 datetime with an explicit timezone — `Z` or `±HH:MM`/`±HHMM`, seconds and fractional seconds optional (e.g. `2026-06-01T20:00:00Z`). Text in any other format is rejected: regional formats (`01/06/2026`) are ambiguous, as are ISO strings without a timezone (UTC vs local). Impossible calendar dates that JS `Date` would silently roll over (e.g. `2026-02-30`) are also rejected.	Converted to ISO 8601 UTC using `date.toISOString();` accepted ISO text is parsed and normalised the same way, so output is identical for both forms.
`"boolean"`	Accepts a native boolean cell or any of the following strings (case-insensitive): `true`/`false`, `yes`/`no`, `y`/`n`, `1`/`0`. Any other value is an error.	Normalised to the lowercase string `"true"` or `"false"`.
`"enum"`	Must be one of the values listed in the `enumValues` array (case-sensitive).	The value as-is.

> **Timezone note:** Date cells in spreadsheets carry no timezone information. ExcelJS interprets them as local time. The `toISOString()` conversion produces a UTC string, which will apply the user's local timezone offset. Users in non-UTC timezones should ensure their date/time values represent the intended UTC moment. ISO 8601 text cells do not have this ambiguity — their explicit timezone designator is honoured (which is why one is required).

Record Validator

An optional `RECORD_VALIDATOR` function can be defined alongside `COLUMN_CONFIG`. It receives the fully resolved field values for a row (after defaults have been applied and dates converted) and returns either `true` (row is valid) or an error string describing the problem.

```
const RECORD_VALIDATOR = (fields) => {
  if (fields["Window Closure Date and Time"] &&
```

```
      fields["Availability Mode"] !== "onDemand") {
    return 'Window Closure Date and Time is only valid when Availability Mode is
        "onDemand"';
  }
  return true;
};
```

The record validator is called after all field-level validation has passed for the row. It is intended for cross-field rules that cannot be expressed as per-field type constraints.

XPath-like Notation

The `xpath` property uses a notation loosely based on XPath but is **not** standard XPath. All paths are relative to the `Publication` element.

- **Attributes** are prefixed with `@`, e.g. `@publicationId` means the `publicationId` attribute on the `Publication` element.
- **Child elements** use `/`-separated paths, e.g. `PublicationDateTime` or `ChannelPlatforms/ChannelPlatform/@label`.
- **Indexed elements** use zero-based bracket notation for repeating elements, e.g. `SubChannel[0]`, `SubChannel[1]`. Each index produces a separate XML element. If the label for an indexed SubChannel is empty/null, that SubChannel element is omitted entirely.

Default Value Handling

- Simple defaults (e.g. `"true"`, `"false"`) are applied as literal string values when the field is unmapped or the cell is empty.
- Function defaults (e.g. `(row) => row["Channel Label"]`) receive the processed row data and return a computed value. Used where one field defaults to the value of another.
- `null` defaults mean no value is inserted and the corresponding XML element/attribute is omitted.

Special XML Generation Rules

1. **SubChannel is mandatory** — the v1 schema requires at least one `<SubChannel>` per `<ChannelPlatform>`. After processing all `COLUMN_CONFIG` fields, the XML generator checks whether a `<SubChannel>` was created. If not, it inserts a default `<SubChannel label="Network" isCore="true"/>`. If sub-channels were provided from the spreadsheet, the first one is marked with `isCore="true"`.
2. **SubChannel elements from the spreadsheet** are only created when at least the label value is present.
3. **ChannelPlatform** requires at least a label.
4. **WindowClosureDateTime** is only included when a value is present. Only semantically valid when `availabilityMode` is `"onDemand"` — enforced by the record validator.

S3 Bucket Structure

```
{bucket}/
├── incoming/       ← newly uploaded XML files land here
├── complete/       ← TEP moves files here after successful processing
└── errors/         ← TEP places error files here on ingestion failure
```

File Naming Convention

```
{prefix}/{sha256_hash}-{batch_id}-{yyyymmdd}-{hhmmss}-{random_number}.xml
```

- `sha256_hash` — 64 lowercase hex characters; SHA-256 of the XML content
- `batch_id` — 8 lowercase hex characters; shared by all files in one upload session, generated via `crypto.getRandomValues`
- `yyyymmdd` — 8-digit date
- `hhmmss` — 6-digit time
- `random_number` — 6-digit zero-padded random number

The batch ID enables grouping files by upload session in the Error Review section. The regex used to validate filenames created by this tool is:

```
/^([a-f0-9]{64})-([a-f0-9]{8})-(\d{8})-(\d{6})-(\d{6})\.xml$/
```

Local Storage Keys

Key	Purpose
`tep_aws_access_key_id`	AWS Access Key ID (also read/written by the key rotation page — see below)
`tep_aws_secret_access_key`	AWS Secret Access Key (also read/written by the key rotation page — see below)
`tep_aws_bucket_name`	S3 bucket name
`tep_aws_region`	AWS region
`tep_upload_hash_cache`	JSON array of up to 1,000 SHA-256 hashes (circular buffer)
`tep_learned_aliases`	JSON object mapping column header strings to field names (e.g. `{"Pub ID": "Publication ID"}`); built up as users confirm non-exact column mappings
`tep_error_cache`	JSON object tracking error review state (batch statuses and cached error details); see Error Review below

---

Access Key Rotation Page

`public/key-rotation.html` is a self-contained page (no framework, no build step, no dependencies) that lets a broadcaster rotate their long-lived IAM access key by calling the AWS IAM API directly from the browser with hand-rolled SigV4 signing. It is served from the uploader's own origin (Vite copies `public/` files into `dist/` verbatim) and is linked, deliberately low-key, from the Settings modal.

Because it shares the uploader's origin, it prefills the key saved in the uploader's settings and, once the replacement key has been verified against IAM, writes the new key back to localStorage **before** the old key is deactivated or deleted — so the uploader switches keys automatically and can never be left holding a dead key. Run from a saved copy (`file://`) or any other origin, it finds no localStorage and behaves as a fully standalone tool with no persistence, which also serves broadcasters who do not use Integration-Lite.

It is intentionally **not** part of the Svelte build so the deployed file stays byte-identical to the audited source. Full design rationale, CORS verification notes and hard constraints are in `keyRotationTool.md` — read that before modifying the page.

Error Review

The Error Review section allows users to browse TEP ingestion results, view error details, download files, and dismiss handled batches.

Architecture

The feature is split across three layers:

Module	Role
`src/lib/s3.js`	S3 operations: `listAllObjects()` (paginated), `downloadObject()`, `parseFilename()`
`src/lib/errorCache.js`	localStorage-backed cache (`tep_error_cache`) for batch status and error detail summaries
`src/lib/errorPipeline.js`	Orchestrator: lists S3, parses filenames, groups by batch ID, reconciles cache, fetches error details

Data flow

1. **List** — `loadErrorBatches()` calls `listAllObjects()` for both `errors/` and `complete/` prefixes in parallel
2. **Parse** — each S3 key is stripped of its prefix and run through `parseFilename()` which validates it against `TEP_FILENAME_REGEX` and extracts the batch ID
3. **Filter** — files not matching the naming convention (i.e. not created by this tool) are discarded, as are files older than 30 days
4. **Reconcile** — the local cache is pruned of entries for files that no longer exist remotely
5. **Group** — files are grouped by batch ID into batch summary objects
6. **Dismiss** — batches the user has previously dismissed are filtered out
7. **Sort** — remaining batches are sorted newest-first by the most recent file's `LastModified`

Lazy detail loading

Error details are not fetched during the initial listing. When the user clicks into a batch, `loadErrorDetail()` is called for each error file:

1. Check `getCachedError(filename)` — return immediately if cached
2. Download `errors/{filename}.json` and `errors/{filename}` (the XML) in parallel via `Promise.allSettled`
3. Parse the JSON to extract: `stage`, error/warning counts, max severity, top error message
4. Parse the XML with `DOMParser` to extract `publicationId` from the `<Publication>` element
5. Cache the summary in localStorage (excluding the full JSON report to save space)
6. Return the detail object to the component for display

Per-file failures are non-fatal — partial data is returned with `null` for missing fields.

Error JSON structure

TEP produces a companion `.json` file for each errored XML file, with the same name plus a `.json` suffix (e.g. `errors/hash-batch-date-time-random.xml.json`):

```
{
  "processingId": "xml-ingestion-bmt-137b9eeb-...",
  "originalFile": "hash-batch-date-time-random.xml",
  "funderSlug": "bmt",
  "timestamp": "2026-05-20T14:45:00.469Z",
  "error": "top-level error summary",
  "stage": "VALIDATING",
```

```
  "errors": [
    { "code": "UNSUPPORTED_XML_NAMESPACE", "message": "...", "severity": "critical",
        "timestamp": "..." }
  ],
  "warnings": [],
  "metrics": {
    "recordsProcessed": 0, "recordsCreated": 0, "recordsUpdated": 0,
    "recordsSkipped": 0, "recordsFailed": 1, "recordsDeleted": 0,
    "parseTime": 1, "validationTime": 1
  }
}
```

Local cache structure (`tep_error_cache`)

```
{
  "batches": {
    "a1b2c3d4": "new|viewed|dismissed"
  },
  "errorDetails": {
    "hash-batch-date-time-random.xml": {
      "stage": "VALIDATING",
      "errorCount": 1,
      "warningCount": 0,
      "maxSeverity": "critical",
      "topError": "...",
      "publicationId": "PUB-001",
      "cachedAt": "2026-05-20T15:00:00Z"
    }
  }
}
```

Cache entries older than 30 days are pruned automatically. Entries for files no longer present in S3 are removed during reconciliation. Batch status is set to `"dismissed"` locally when the user clicks “Dismiss batch” — since the app does not have delete permissions on the `errors/` prefix, a bucket lifecycle policy removes the remote files after 14 days.

Component architecture

```
ErrorReview.svelte          ← section root: credentials check, loading, view routing
  ErrorBatchList.svelte     ← batch list with refresh, empty state
    ErrorBatchCard.svelte   ← single batch row: ID, timestamp, error/OK counts, dismiss
  ErrorBatchDetail.svelte   ← detail view: lazy-loads error details, file download
    ErrorFileRow.svelte     ← expandable <details> row per error file
```

Stores

Store	Type	Purpose
`errorReviewView`	`writable('list')`	Sub-navigation: `'list'` or `'detail'`
`errorReviewBatchId`	`writable(null)`	Batch ID currently being viewed in detail

These are stores (not component-local state) so the user’s position is preserved across tab switches.

Learned Aliases

To reduce repetitive manual column mapping, the app persists confirmed mappings to `localStorage`.

Storage

Key: `tep_learned_aliases` Format: a flat JSON object where each key is a column header string (as it appeared in the spreadsheet) and each value is the `fieldName` from `COLUMN_CONFIG`:

```
{
  "Pub ID": "Publication ID",
  "Ep Identifier": "Episode ID",
  "Air Date": "Publication Date and Time"
}
```

How aliases are saved (`learnedAliases.js`)

`saveLearnedAlias(columnHeader, fieldName)` is called from `ColumnMappingReview.svelte` when the user clicks **Proceed**. It:

1. Finds the `COLUMN_CONFIG` entry for `fieldName`.
2. Checks whether the column header already exists in the field's hard-coded `aliases` array (case-insensitive). If it does, the function is a no-op.
3. Otherwise writes `{ columnHeader: fieldName }` into the stored object.

How aliases are used (`mapping.js`)

`autoMapColumns()` calls `getLearnedAliases()` at the start of its loop. For each spreadsheet column header:

1. Lowercased header is looked up in the learned aliases map first.
2. If a match is found for an unassigned field, that field is assigned at **distance 0** and the column is skipped for Levenshtein matching.
3. Distance 0 on all fields satisfies the auto-skip conditions (Step 6), so subsequent uploads with the same spreadsheet structure skip the Column Mapping Review entirely.

Clearing learned aliases

Learned aliases are cleared when the user clears their browser's site data (`localStorage`). No explicit UI is provided for clearing them — if a mapping becomes wrong, the user can correct it on the Column Mapping Review screen and the corrected header will overwrite the old entry on the next Proceed.

Debug Mode

Debug mode is activated by appending `?debug=true` to the URL. The flag is read once on load and stored as a Svelte readable store (`debugMode`).

Console logging

`lib/debug.js` exports a `log(...args)` utility that calls `console.log` only when `debugMode` is true and is a no-op otherwise. All significant app events should use `log()` rather than bare `console.log` — step transitions, column mapping decisions, validation results, S3 calls, hash computations, etc.

XML preview

After Step 9 (XML generation) and before upload begins, if `debugMode` is true, an additional **"Preview XML"** button is shown alongside the normal "Proceed" button. Clicking it opens a new browser window and uses `document.write` to render a simple HTML page displaying all generated XML documents in formatted `<pre>` blocks (one per row), allowing the full XML output to be inspected without uploading anything.

Debug mode has no effect in normal usage.

Dependencies

Dependency	Source
Svelte	npm (build-time)
Vite	npm (build-time)
ExcelJS	npm
AWS SDK for JavaScript v3 (S3 client)	npm
Levenshtein distance	npm (e.g. `fastest-levenshtein`)
Web Crypto API	browser-native; no dependency